

RECRUITAI

PROJECT SPECIFICATION & IMPLEMENTATION BLUEPRINT

Event Planner Management System

Detailed project brief generated from the project record.

CANDIDATE

Varun Chopra

DURATION

14 days

TECHNOLOGY STACK

Java

Spring Boot

Angular

TypeScript

MySQL

REST API

Project Overview

Summary of the problem, purpose and expected outcome.

PROJECT SUMMARY

A web application that lets organizers create and manage events, send invitations, track RSVPs, and view simple analytics. Users register, log in, and manage their own events through an Angular UI backed by a Spring Boot REST API and MySQL storage.

CANDIDATE Varun Chopra	TYPE fullstack	DURATION 14 days
--	--------------------------------------	--

Primary Objectives

- Implement a secure JWT based authentication system.
- Expose CRUD REST endpoints for event management.
- Build an Angular SPA with forms, routing, and state management.
- Integrate email invitation sending via a mock service.
- Provide unit and integration tests for backend and frontend.
- Deploy the application to a local Docker environment.

Candidate Details

NAME

Varun Chopra

Technology Stack

Technologies, frameworks and tools expected for implementation.

Java

Technology / Tool

Spring Boot

Technology / Tool

Angular

Technology / Tool

TypeScript

Technology / Tool

MySQL

Technology / Tool

REST API

Technology / Tool

Functional Requirements

- Register: POST /api/auth/register – creates a new user with hashed password.
- Login: POST /api/auth/login – returns a JWT on successful authentication.
- Create Event: POST /api/events – stores event details (title, date, location).
- Update Event: PUT /api/events/{id} – modifies existing event.
- Delete Event: DELETE /api/events/{id} – removes an event.
- Send Invitation: POST /api/events/{id}/invite – records invitation and triggers email.
- RSVP: POST /api/invitations/{id}/rsvp – records participant response.
- Dashboard: GET /api/dashboard – returns summary statistics for the logged in organizer.

Non-Functional Requirements

- Passwords must be stored using BCrypt hashing.
- JWT tokens must expire after 1 hour.
- API responses must follow standard HTTP status codes.
- Frontend must be responsive down to 320 px width.
- Application must run on Docker with separate containers for backend, frontend, and database.

Modules & Features

Major functional areas that make up the project.

Authentication

Handles user registration, login, JWT issuance, and role checks.

- User registration with validation
- Login with password verification
- JWT generation and validation middleware
- Password reset (mock)
- Role based access control

Event Management

Core CRUD operations for events owned by a user.

- Create, read, update, delete events
- Event date validation
- Owner only access enforcement

- Pagination of event list
- Search by title

Invitation & RSVP

Manages sending invitations and tracking participant responses.

- Add participants by email
- Send invitation email (mock service)
- Record RSVP status (Accepted/Declined)
- Prevent duplicate invitations
- Notify organizer of RSVP changes

Reporting Dashboard

Provides summary metrics for the organizer.

- Total events count
- Upcoming events list
- RSVP acceptance rate
- Export summary as CSV
- Responsive chart widgets

Implementation Roadmap

Recommended phases and milestones for completing the project.

1

Foundation

Initialize Git repo with README, Create Spring Boot starter project, Create Angular workspace, Write Dockerfiles for backend and frontend, Compose MySQL service, Configure CI script placeholder

- Initialize Git repo with README
- Create Spring Boot starter project
- Create Angular workspace
- Write Dockerfiles for backend and frontend
- Compose MySQL service

2

Core Backend

Design MySQL schema (User, Event, Role), Implement JPA entities and repositories, Add BCrypt password encoder, Create AuthController with register/login, Create EventController with CRUD endpoints, Write JUnit tests for Auth and Event services

- Design MySQL schema (User, Event, Role)
- Implement JPA entities and repositories
- Add BCrypt password encoder
- Create AuthController with register/login
- Create EventController with CRUD endpoints

3

Frontend MVP

Set up Angular routing and guards, Create AuthService for JWT handling, Build LoginComponent and RegisterComponent, Develop EventListComponent with pagination, Integrate backend APIs via HttpClient, Add basic responsive styling

- Set up Angular routing and guards
- Create AuthService for JWT handling
- Build LoginComponent and RegisterComponent
- Develop EventListComponent with pagination
- Integrate backend APIs via HttpClient

4

Intermediate Features

Create InvitationService and RSVPService, Build InviteComponent with email input, Implement RSVPComponent for participants, Develop DashboardComponent with charts, Mock email service integration, Write integration tests for new endpoints

- Create InvitationService and RSVPService
- Build InviteComponent with email input
- Implement RSVPComponent for participants
- Develop DashboardComponent with charts
- Mock email service integration

5

Quality & Deployment

Add end to end Cypress tests for critical flows, Write API documentation (README and Swagger), Create Docker compose production profile, Configure environment variables for JWT secret, Run performance sanity check (load 10 concurrent users), Prepare final demo video

- Add end to end Cypress tests for critical flows
- Write API documentation (README and Swagger)
- Create Docker compose production profile
- Configure environment variables for JWT secret
- Run performance sanity check (load 10 concurrent users)

API Design

Key API endpoints and expected responsibilities.

POST

/api/auth/register

Create new user

POST**/api/auth/login**

Authenticate and receive JWT

Implementation Note

This document is generated from the project record. AI-generated requirements should be reviewed by an administrator before implementation.